

Docker & CI/CD for Kedro-Python Projects: a pragmatic guide

1 Mental model – Docker for Python/ML projects

What Docker is (image vs container)

Docker packages a filesystem and execution environment so your code runs the same everywhere. A **Docker image** is a read-only template created from a `Dockerfile`. When you start it, Docker creates a **container**: a sandboxed process with its own filesystem, network namespace and PID tree. Using images makes ML experiments reproducible because you can rebuild exactly the same environment in CI and production. Official guidance stresses using multi-stage builds to keep images small and reproducible by separating build and runtime stages ¹.

What belongs in the image vs mounted at runtime

The image should contain **code** (e.g., your `src/` package), **locked dependencies** (`requirements.txt`, `pyproject.lock`, etc.), basic **Kedro configuration** (`conf/base/`), and sensible defaults. Things that change per environment or shouldn't be baked into the image must be mounted:

- **Data and model artifacts** – `data/` can be huge and often contains secrets; Docker best-practices advise excluding data from the build context via `.dockerignore` ².
- **Secrets and credentials** – never commit them or bake them into the image. Instead inject them with environment variables or BuildKit secret mounts. GitHub's secret documentation warns against passing secrets on the command line and recommends using environment variables or STDIN to avoid leaks ³.
- **Environment-specific configs** – `conf/local/` or `conf/<env>/` in Kedro override `conf/base/`. The Kedro `ConfigLoader` merges base then environment-specific config and environment paths loaded later override earlier ones ⁴. Use `--env <env>` or the `KEDRO_ENV` variable to load additional envs ⁵.

Inner loop vs outer loop

Two loops drive development:

- **Local development loop**: you build quickly, run the container with source mounted, run unit tests/lint, and iterate. Build images sparingly; rely on volume mounts and interactive shells.
- **CI loop**: the pipeline builds the image once using cache layers, runs tests/lint inside the same environment and optionally runs a small pipeline, then pushes the image to a registry. Docker's multi-stage builds and BuildKit caches keep CI fast and reproducible ⁶.

How Kedro changes the picture

Kedro projects have a conventional structure. A typical repo contains:

- `src/<package_name>/` – your Python package with pipelines and nodes.
- `conf/base/` – base configuration (`catalog.yml`, `parameters.yml`) used in all environments.
- `conf/local/` – machine-specific overrides; this folder often contains credentials and should not be committed ⁷. Additional environments can be created (e.g., `conf/prod/`) and selected with `--env` ⁵.
- `data/01_raw`, `data/02_intermediate`, etc. – layered data folders; these contain datasets and should not be baked into the image ⁸.

Because data is external, the image should mount `data/` at runtime. When running `kedro run`, you can choose a pipeline via `--pipeline` (omitting it runs the default pipeline ⁹) and specify an environment via `--env` (defaults to `local`) ¹⁰.

2 Repository essentials (Docker-ready structure)

An opinionated checklist:

Item	Guidance
<code>.dockerignore</code>	Exclude <code>data/</code> , <code>.git/</code> , <code>.venv/</code> , <code>__pycache__</code> , <code>logs/</code> , <code>*.egg-info</code> , <code>dist/</code> , <code>*.pyc</code> , IDE folders, caches and any secrets. Smaller build context speeds up builds and prevents accidental leaks ¹¹ .
Secrets	Store secrets in <code>.env</code> for local use only. Never commit <code>.env</code> ; put it in <code>.gitignore</code> and <code>.dockerignore</code> . In CI, set secrets in GitHub and inject via environment variables or BuildKit secret mounts ³ .
Folder layout	Keep <code>Dockerfile</code> at repo root or under <code>docker/</code> . Optionally place compose files in <code>docker/compose.yml</code> to keep things tidy.
Packaging	Use modern packaging (e.g., <code>pyproject.toml</code> with <code>uv</code> , <code>pip</code> , or <code>poetry</code>). Pin dependencies and generate a lock file.
Version control	Check in only source code and config; do not commit data or credentials.

Minimal Kedro repo structure:

```
my_project/
├── .dockerignore
├── Dockerfile
├── compose.yml          # optional Docker Compose for dev
├── pyproject.toml or requirements.txt
├── src/my_package/
└── conf/base/
```

```
|— conf/local/      # ignored by git
|— data/           # mounted at runtime, ignored in image
|— tests/
```

3 Minimal Docker setup

`.dockerignore` **example**

```
# ignore data and secrets
data/
conf/local/
.env
.aws/
.ssh/

# ignore build artifacts
__pycache__/
.mypy_cache/
*.egg-info/
dist/

# ignore version control and IDE files
.git/
.vscode/
.idea/
```

Multi-stage Dockerfile

The following Dockerfile uses two stages: **builder** (installs dependencies) and **runtime** (contains only necessary files). It runs as a non-root user and caches layers effectively by copying lock files before source code ¹ ² .

```
# syntax=docker/dockerfile:1

##### Builder stage #####
FROM python:3.11-slim as builder

# Install build tools only here
RUN apt-get update && apt-get install -y build-essential && rm -rf /var/lib/apt/
lists/*

WORKDIR /app

# Copy only dependency files first (improves caching)
```

```

COPY pyproject.toml poetry.lock requirements.txt* ./

# Create virtual environment in /venv for caching
RUN python -m venv /venv
ENV PATH="/venv/bin:$PATH"

RUN pip install --upgrade pip && \
    # choose your installer: pip/uv/poetry
    ( [ -f poetry.lock ] && pip install poetry && poetry export -f
requirements.txt --output requirements.lock || true ) && \
    pip install --no-cache-dir -r requirements.lock

# Copy the rest of the source
COPY src/ src/
COPY conf/base/ conf/base/

##### Runtime stage #####
FROM python:3.11-slim as runtime

# Create non-root user
RUN useradd -m appuser
WORKDIR /app

# Copy virtualenv from builder
COPY --from=builder /venv /venv
ENV PATH="/venv/bin:$PATH"

# Copy source and config
COPY src/ src/
COPY conf/base/ conf/base/

# Avoid running as root
USER appuser

# Default command runs Kedro; override with your own cmd in docker run
CMD ["kedro", "run"]

```

compose.yaml for local development

Use Docker Compose to develop quickly: mount source code and data, pass environment variables, and optionally start dependencies (e.g., PostgreSQL). Replace `my_project`, `my_package`, and adjust ports as needed.

```

services:
  kedro:
    build: .

```

```

image: my_project:dev
command: kedro run
environment:
  - KEDRO_ENV=local    # choose configuration environment
  - PYTHONUNBUFFERED=1
volumes:
  - ./:/app:cached      # mount source for live reload
  - ./data:/app/data:rw # mount data directory
  - ./conf/local:/app/conf/local:ro # optional local overrides
ports:
  - "8888:8888" # e.g., for kedro viz or notebooks

# Optional service (e.g., PostgreSQL)
# postgres:
#   image: postgres:16
#   environment:
#     POSTGRES_PASSWORD: example
#   ports:
#     - "5432:5432"

```

4 Day-to-day workflows

Build the image

```

# Build with BuildKit caching
DOCKER_BUILDKIT=1 docker build -t my_project:dev .

```

Run the container (development)

```

# Mount source and data for interactive development
docker run --rm -it -v "$PWD":/app -v "$PWD/data":/app/data \
  -e KEDRO_ENV=local my_project:dev /bin/bash

```

Run unit tests and lint inside the container

```

# Example uses pytest and ruff; adjust commands as needed
docker run --rm -v "$PWD":/app my_project:dev \
  bash -c "pytest -q && ruff src/"

```

Run a Kedro pipeline

```
# Run the default pipeline (defined in register_pipelines)
docker run --rm -v "$PWD":/app -v "$PWD/data":/app/data \
  -e KEDRO_ENV=local my_project:dev kedro run

# Run a named pipeline
docker run --rm -v "$PWD":/app -v "$PWD/data":/app/data \
  my_project:dev kedro run --pipeline=data_science

# Pass parameters or different env
docker run --rm -v "$PWD":/app -v "$PWD/data":/app/data \
  -e KEDRO_ENV=prod my_project:dev kedro run --pipeline=model_training \
  --params="{ 'model_depth': 5 }"
```

Debugging and fixes

- **View logs:** use `docker logs <container>` or set verbose logging (`kedro run -v`).
- **Check installed packages:** run `docker run --rm my_project:dev pip list` or `python -V`.
- **Fix “it works on my machine”:** ensure dependencies are pinned and environment variables are identical in local and CI. Clean your Docker cache if builds behave oddly: `docker builder prune`.

5 CI/CD with GitHub Actions

Pipeline overview

1. **Pull requests:** build the image with BuildKit caching, run lint and tests inside the container, optionally run a small Kedro pipeline using synthetic data to catch integration errors. Do not push images on PRs.
2. **Release tags (e.g., `v1.2.3`) or pushes to main:** build the production image, tag it with `sha-
<short_sha>` and semantic version tags, log in to GitHub Container Registry (GHCR) or your registry, push the image, and optionally deploy.
3. **Caching:** Use `docker/setup-buildx-action` and `cache-from`/`cache-to` parameters to speed up builds by reusing layers ¹². Use BuildKit inline cache for PR builds and a registry cache for release builds (e.g., `cache-to: type=registry,ref=user/app:buildcache,mode=max` ¹³).
4. **Secrets:** Store registry credentials and other secrets as GitHub Secrets. Inject them via the `secrets` input to `docker/build-push-action` or environment variables. Never pass secrets on the command line ³.
5. **Environment-specific deploys:** Use GitHub environments (dev/staging/prod) with required reviewers to protect deployments.

`.github/workflows/ci.yml` – PR checks

```
name: ci
```

```
on:
```

```

pull_request:
  branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    env:
      REGISTRY: ghcr.io
      IMAGE_NAME: ${ env.github.repository }
    steps:
      - uses: actions/checkout@v5

      # set up Python to run tests outside the container (optional)
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install Python dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run lint and unit tests
        run: |
          ruff src/
          pytest -q

      # Build docker image but do not push; use cache for speed
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Build image for tests
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true          # makes image available for docker run
          tags: ${ env.IMAGE_NAME }:pr-${ env.github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Run Kedro smoke test
        run: |
          docker run --rm ${ env.IMAGE_NAME }:pr-${ env.github.sha } \
            kedro run --pipeline=__default__

```

This workflow builds the image once, runs a Kedro smoke test inside the container, and ensures lint/tests pass. It uses GitHub Actions cache exporter (`type=gha`) to speed up subsequent builds ¹⁴ .

`.github/workflows/release.yml` - release & push

```
name: release

on:
  push:
    tags:
      - 'v*.*.*' # semantic version tags

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
      id-token: write
    steps:
      - uses: actions/checkout@v5

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Extract metadata (tags & labels)
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${ env.REGISTRY }/${ env.IMAGE_NAME }

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
```



```

    context: .
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    cache-from: type=registry,ref=${{ env.REGISTRY }}/$
    {{ env.IMAGE_NAME }}:buildcache
    cache-to: type=registry,ref=${{ env.REGISTRY }}/$
    {{ env.IMAGE_NAME }}:buildcache,mode=max

- name: Create artifact attestation (optional)
  uses: actions/attest-build-provenance@v3
  with:
    subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
    subject-digest: ${{ steps.build-and-push.outputs.digest }}
    push-to-registry: true

```

This workflow runs on tag pushes. It logs into GHCR using `docker/login-action`¹⁵, generates tags and labels via `docker/metadata-action`¹⁶, builds and pushes the image, and exports the cache to the registry for future builds¹⁷. Tag strategy uses semantic version tags (e.g., `v1.2.3`), the Git commit SHA (via metadata action), and optionally `latest` for default branch. Permissions are set to allow writing to the packages registry¹⁸.

Required secrets and permissions

- `GITHUB_TOKEN` (automatic) – used to authenticate pushes to GHCR.
- **Registry credentials** – if pushing to Docker Hub or another registry, store `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` secrets and pass them to `docker/login-action`.
- **Additional secrets** – such as database passwords, API keys, etc., should be stored in repository or organization secrets and passed via environment variables or BuildKit secret mounts as shown above¹⁹.

6 Do this / Don't do this

□ Do this

- **Pin dependencies and lock versions** for reproducible builds. Use a `requirements.lock` or `poetry.lock` file. Avoid `pip install -U` in CI.
- **Use** `.dockerignore` to exclude large files, caches and secrets¹¹.
- **Use multi-stage builds** to keep images slim and separate build tools from runtime¹.
- **Run as a non-root user** by adding a user in the Dockerfile².
- **Inject secrets at runtime** via environment variables or BuildKit secrets; rotate them regularly³.
- **Use BuildKit caching** (`cache-from` / `cache-to`) to speed up CI builds¹².
- **Use GitHub environments** with required reviewers to gate production deployments.

❏ Don't do this

- **Don't bake data or credentials into your image.** Keep `data/` and `conf/local/` out of the build context and mount them at runtime ¹¹.
- **Don't run as root** inside containers. Use non-root user to reduce attack surface ².
- **Don't install build tools in the runtime stage.** Use multi-stage builds so runtime images are lean.
- **Don't copy gigabytes of data into the build context.** Use `.dockerignore` to avoid slow builds and large images ¹¹.
- **Don't pass secrets on the command line.** Use environment variables or BuildKit secret mounts ³.
- **Don't forget health checks and logging.** Add simple health checks (e.g., run `kedro run --help` in CI) and use proper logging drivers or volume mounts for logs.

7 Troubleshooting – fast fixes

- **Slow builds** – ensure `.dockerignore` excludes large files and caches. Use `cache-from` and `cache-to` settings. If using BuildKit, set `DOCKER_BUILDKIT=1`.
- **ModuleNotFoundError inside container** – ensure your `src/` is a Python package (contains `__init__.py`) and is added to `PYTHONPATH`. In the Dockerfile, install your package (`pip install -e .`) or set `PYTHONPATH=/app/src`.
- **Kedro config or credentials missing** – mount `conf/local/` and set `KEDRO_ENV` accordingly. Use `--env` flag to select the right config ¹⁰.
- **File permissions** – when running as non-root, volumes mounted from host may have root ownership. Adjust by setting proper UID/GID in the Dockerfile or using `--user $(id -u):$(id -g)` when running `docker run`.
- **Platform mismatch** – building on Apple Silicon may produce `arm64` images that cannot run on AMD64. Use Buildx with `platforms: linux/amd64` and `linux/arm64` in CI to build multi-platform images.
- **Cache invalidation** – if changes aren't picked up, run `docker builder prune` to clear stale caches.

8 One-page cheatsheet

Docker commands

- Build with cache: `DOCKER_BUILDKIT=1 docker build -t my_project .`
- Run container: `docker run --rm -it -v $(pwd):/app my_project /bin/bash`
- View logs: `docker logs <container>`
- Exec into running container: `docker exec -it <container> bash`
- Clean up: `docker system prune` / `docker builder prune`.

Kedro-in-Docker commands

- Run default pipeline: `docker run --rm my_project kedro run`
- Run named pipeline: `docker run --rm my_project kedro run --pipeline=<name>`
- Select config env: `-e KEDRO_ENV=prod`
- Pass parameters: `--params="{ 'p': 1 }"`

GitHub Actions checklist

- Use `actions/checkout@v5`, `docker/setup-buildx-action@v3`, and `docker/build-push-action@v6`.
- Cache layers: `cache-from: type=gha` and `cache-to: type=gha,mode=max` ¹⁴.
- Use `docker/metadata-action` for tag generation ¹⁶.
- Login to registry with `docker/login-action` ¹⁵.
- Store secrets in GitHub Secrets and reference them via `${{ secrets.NAME }}` ³.

Tagging strategy

1. Semantic version tag (`v1.2.3`) triggers release workflow.
2. Use `sha-<shortsha>` as an immutable tag for each build.
3. Optionally tag `latest` only on the default branch.
4. Push build cache to `<image>:buildcache` and reuse it for subsequent builds.
5. Keep old tags for traceability; avoid moving tags to different images.

Minimal file map

```
.dockerignore      # excludes data, secrets, caches
Dockerfile         # multi-stage build, non-root user
compose.yaml       # dev mode: mount code and data
.github/workflows/ci.yml      # PR checks: build, lint, test, smoke pipeline
.github/workflows/release.yml # Release: build & push to GHCR
src/my_package/    # Python package with Kedro pipelines
conf/base/         # base config (catalog.yml, parameters.yml)
conf/local/        # local overrides (ignored by git)
data/              # external data; mounted at runtime
```

¹ ² Best practices | Docker Docs

<https://docs.docker.com/build/building/best-practices/>

³ Using secrets in GitHub Actions - GitHub Docs

<https://docs.github.com/en/actions/how-to/write-workflows/choose-what-workflows-do/use-secrets>

⁴ ⁵ Configuration — Kedro 0.17.7 documentation

https://docs.kedro.org/en/0.17.7/04_kedro_project_setup/02_configuration.html

⁶ ¹² ¹³ ¹⁴ ¹⁷ Cache management | Docker Docs

<https://docs.docker.com/build/ci/github-actions/cache/>

⁷ Ben Hayes - Building Data Pipelines with Kedro

<https://benhay.es/posts/building-pipelines-kedro/>

⁸ Kedro for Data Pipelines: A Quick Guide | by Shikha Singhal | Dec, 2025 | Medium

<https://medium.com/@singhalshikha111/as-data-scientists-we-often-start-projects-inside-jupyter-notebooks-4994a9f282f8>

9 Run a pipeline — kedro 0.18.14 documentation

https://docs.kedro.org/en/0.18.14/nodes_and_pipelines/run_a_pipeline.html

10 Kedro CLI -

https://docs.kedro.org/en/stable/getting-started/commands_reference/

11 Docker Best Practices for Python Developers | TestDriven.io

<https://testdriven.io/blog/docker-best-practices/>

15 16 18 Publishing Docker images - GitHub Docs

<https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images>

19 Build secrets | Docker Docs

<https://docs.docker.com/build/ci/github-actions/secrets/>